

COMPENDIO E · ALGORITMIA, PSEUDOCODIGO Y POO

Diagramas de flujo, pseudocodigo, prueba de escritorio, programacion OO, lenguajes y tendencias web

Alcance de este documento: síntesis de estudio propia elaborada para la preparación del concurso. **NO reproduce ni reemplaza las obras originales** de la bibliografía oficial del Programa Intelectual de Ingreso del CMN (obras protegidas por derecho de autor de sus respectivos editores). Para el texto completo consulte los originales indicados en la bibliografía.

1. Algoritmos y resolución de problemas

Un **algoritmo** es una secuencia finita y ordenada de pasos, sin ambigüedad, que resuelve un problema. Características: **preciso** (orden claro), **definido** (mismas entradas, mismo resultado) y **finito** (termina). Toda solución tiene **entrada**, **proceso** y **salida**.

- **Etapas:** análisis (entradas y salidas) → diseño (diagrama de flujo o pseudocódigo) → codificación → prueba y depuración → documentación y mantenimiento.

2. Diagramas de flujo (ANSI / ISO 5807)

Símbolo	Forma	Uso
Terminal	Ovalo o rectángulo redondeado	Inicio y Fin
Proceso	Rectángulo	Cálculo o asignación
Entrada / Salida	Paralelogramo	Leer o mostrar datos
Decision	Rombo	Condición con exactamente dos salidas (V/F)
Conector	Círculo	Une partes en la misma página
Conector de página	Pentágono	Continúa en otra hoja
Subproceso	Rectángulo con barras laterales	Módulo definido aparte
Documento / Pantalla	Hoja ondulada / monitor	Salida impresa / por pantalla

- **Reglas:** un único inicio y un único fin; flujo de arriba hacia abajo y de izquierda a derecha; las líneas no se cruzan (se usan conectores); todo símbolo salvo el fin tiene salida.

3. Pseudocódigo (estilo PSeInt) y prueba de escritorio

Sintaxis	Significado
Algoritmo nombre ... FinAlgoritmo	Inicio y fin del programa
Definir x Como Entero / Real / Caracter / Logico	Declaración de variables
x <- expresión	Asignación
Leer a / Escribir "texto", x	Entrada y salida
+ - * / ^ MOD = <> < > <= >= Y O NO	Operadores (MOD = resto)
Si c Entonces ... SiNo ... FinSi	Selección simple y doble
Segun x Hacer 1: ... De Otro Modo: ... FinSegun	Selección múltiple
Mientras c Hacer ... FinMientras	Repetición: pregunta antes (0 o más vueltas)
Repetir ... Hasta Que c	Repetición: pregunta después (1 o más vueltas)
Para i <- 1 Hasta n Con Paso 1 Hacer ... FinPara	Repetición con contador

- **Patrones:** contador ($c \leftarrow c + 1$), acumulador ($s \leftarrow s + x$; el de producto arranca en 1) y bandera (variable lógica). Se inicializan antes del bucle.

```

Algoritmo Factorial
  Leer n
  f <- 1
  Para i <- 1 Hasta n Hacer
    f <- f * i
  FinPara

```

Escribir f
FinAlgoritmo

Prueba de escritorio (n = 4)	i	f	Detalle
Inicio	-	1	Acumulador de producto inicializado en 1
Vuelta 1	1	1	1 x 1
Vuelta 2	2	2	1 x 2
Vuelta 3	3	6	2 x 3
Vuelta 4	4	24	6 x 4 → Escribir muestra 24

4. Programacion estructurada y orientada a objetos

- **Bohm-Jacopini (1966)**: todo algoritmo se expresa con secuencia, seleccion e iteracion. **Dijkstra (1968)**: abandonar el GOTO. Diseño descendente, alta cohesion y bajo acoplamiento.
- **POO**: clase (molde) y objeto (instancia) con atributos y metodos. Pilares: **abstraccion**, **encapsulamiento**, **herencia** y **polimorfismo**. Pioneros: Simula 67 y Smalltalk.
- **UML (clases)**: visibilidad + publico, - privado, # protegido; agregacion (rombo vacio), composicion (rombo lleno), herencia (triangulo vacio), dependencia (flecha punteada).

5. Ejemplos de lenguajes y nuevas tendencias web

Lenguaje	Año	Paradigma / uso
FORTRAN	1957	Imperativo · calculo científico
LISP	1958	Funcional · IA
COBOL	1959	Imperativo · gestion comercial
Pascal	1970	Estructurado · enseñanza
C	1972	Imperativo · sistemas (UNIX)
Prolog	1972	Logico
C++	1983	Multiparadigma / OO
Python	1991	Multiparadigma · datos y automatizacion
Java	1995	OO · bytecode y JVM
JavaScript	1995	Multiparadigma · web en el cliente
PHP	1995	Web del lado del servidor
C#	2000	OO · plataforma .NET

- **Tendencias**: SPA (React, Angular, Vue), PWA (service worker, instalable, offline), renderizado en servidor / generacion estatica, APIs REST y GraphQL con JSON, microservicios y serverless, WebAssembly, TypeScript, diseño responsive y seguridad por defecto (HTTPS).

Fuente: Modelo de Examen Escrito - Analista de Sistemas (CMN), puntos a) y b), que agregan estos temas a los del Programa Intelectual de Ingreso.